INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA
FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

EVIDENCIA DE APRENDIZAJE 2 (EA2):
SISTEMA AUTOMATIZADO DE CLASIFICACIÓN DE PAQUETES

Lógica y Control de Flujo en C++ para Embalaje Industrial en Línea Continua

Estudiante:	Jhoan Esteban Pérez Herrera
Programa Académico:	Ingeniería Mecatrónica
Asignatura:	Programación Orientada a Objetos (Ruta C++)
Docente Titular:	Juan Esteban Acevedo Carvajal
Institución:	IU Digital de Antioquia
Fecha:	Noviembre de 2026

1. Introducción y Alcance del Proyecto

El presente informe documenta el diseño, modelado, implementación y validación de una solución algorítmica en lenguaje C++ para la automatización de la estación de embalaje y enrutamiento de paquetes en una línea continua de manufactura. El desarrollo se enfoca en resolver un problema real del entorno productivo mediante estructuras fundamentales de control de flujo (variables primitivas, condicionales lógicos y ciclos iterativos controlados), asegurando una arquitectura robusta, eficiente y libre de sobreingeniería.

2. Contexto Territorial y Definición del Problema

En continuidad con el escenario evaluado en la Evidencia de Aprendizaje 1 (EA1), la problemática se sitúa en una fábrica de repuestos mecánicos e industriales ubicada en la zona industrial de Itagüí (Valle de Aburrá, Antioquia). La planta produce piezas metálicas que son embaladas en cajas rectangulares y despachadas mediante dos bandas transportadoras diferenciadas: la Banda A, destinada a paquetes de gran volumen para distribución de carga pesada, y la Banda B, asignada a cajas compactas o de tamaño estándar.

> **Clasificación y Naturaleza de los Datos:**
> • Hechos del Proceso: Las cajas poseen geometría rectangular y deben clasificarse según su volumen cúbico. Existen dos bandas de evacuación física (Banda A para paquetes mayores a 500 cm^3 y Banda B para paquetes menores o iguales a 500 cm^3).
> • Datos Simulados: El sistema captura por consola las tres dimensiones del paquete (ancho, alto y profundidad en centímetros), simulando las lecturas que en una etapa de automatización física proporcionarían sensores ópticos o de ultrasonido.
> • Supuestos Operativos: Se asume que las medidas representan dimensiones reales en centímetros, que valores menores o iguales a cero corresponden a fallas de sensado o errores humanos que deben descartarse, y que la producción se procesa en lotes continuos hasta que el operario detenga la sesión.

Justificación del Control de Flujos (Decisiones y Ciclos):
1. Estructuras de Decisión (if/else): Son indispensables para dos propósitos críticos: en primer lugar, validar la integridad física de las lecturas, bloqueando valores menores o iguales a cero antes de cualquier operación; en segundo lugar, evaluar el umbral volumétrico de 500 cm^3 para accionar el desvío neumático correspondiente hacia la Banda A o la Banda B.
2. Estructura Repetitiva (Ciclo do-while): Una línea de ensamblaje y embalaje trabaja de manera secuencial e ininterrumpida. El ciclo do-while garantiza que el programa procese al menos una primera pieza al iniciar turno y permanezca en ejecución mientras haya material en la banda, consolidando métricas totales de producción al finalizar la jornada.

3. Modelado Algorítmico: Entradas, Procesos y Salidas

La lógica del sistema se estructura mediante la siguiente matriz operacional de Entradas, Procesos y Salidas (E-P-S), asociando cada componente a su correspondiente tipo de dato y estructura en C++:

Componente	Variable / Estructura C++	Descripción Funcional
Entradas	float ancho, alto, profundidad;	Lectura de dimensiones de la caja en centímetros ingresadas por consola.
Control de Lote	char continuar; do { ... } while (...);	Controla la ejecución repetitiva del lote hasta que el usuario decida parar.
Filtro de Integridad	if (ancho <= 0 || alto <= 0 || profundidad <= 0)	Descarta dimensiones físicas no válidas sin alterar los contadores del lote.

Cálculo y Acumulación	volumen = ancho * alto * profundidad;
	volumenTotal += volumen;
	totalCajas++;
		Cálculo matemático del volumen cúbico y actualización de acumuladores del turno.

Enrutamiento	if (volumen > 500.0f) ... else ...	Decisión lógica para desviar la caja a Banda A (> 500 cm^3) o a Banda B (<= 500 cm^3).

4. Implementación del Código Fuente en C++

El algoritmo fue programado en C++ estándar dentro del archivo clasificador_cajas.cpp, priorizando la legibilidad, la robustez ante datos anómalos y la eficiencia operativa:

```
// ================================================================================
// INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA - IU DIGITAL
// Asignatura: Programación Orientada a Objetos (Ruta Ingeniería Mecatrónica)
// Evidencia de Aprendizaje 2 (EA2): Modeladores de Flujos: Lógica y Control en el Entorno
//
// Estudiante: Jhoan Esteban Pérez Herrera
// Docente: Juan Esteban Acevedo Carvajal
//
// Contexto Plausible: Automatización de la línea de embalaje y clasificación de cajas
// en una fábrica de repuestos industriales en el Valle de Aburrá.
// ================================================================================

#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    // --------------------------------------------------------------------------
    // 1. DECLARACIÓN DE VARIABLES Y TIPOS DE DATOS
    // --------------------------------------------------------------------------
    // Variables para almacenar las medidas de la caja ingresadas por consola
    float ancho = 0.0f;
    float alto = 0.0f;
    float profundidad = 0.0f;
    float volumen = 0.0f;

    // Variables acumuladoras y contadores de control para el lote continuo
    int totalCajas = 0;
    int cajasBandaA = 0;
```

```
    int cajasBandaB = 0;
    float volumenTotal = 0.0f;

    // Variable para controlar la condición de terminación del ciclo
    char continuar = 'S';

    cout << "============================================================" << endl;
    cout << "   SISTEMA DE CLASIFICACION AUTOMATIZADA DE CAJAS        " << endl;
    cout << "     Fabrica de Repuestos - Valle de Aburra (EA2)        " << endl;
    cout << "============================================================" << endl;

    // ---------------------------------------------------------------------
    // 2. ESTRUCTURA REPETITIVA (CICLO do-while)
    // Se utiliza un ciclo do-while porque el proceso en la banda transportadora
    // debe ejecutar al menos una lectura de caja y continuar mientras el
    // operario confirme que hay más piezas en el turno (continuar == 'S' o 's').
    // ---------------------------------------------------------------------
    do {
        cout << "\n--- Registro de Caja #" << (totalCajas + 1) << " ---" << endl;

        cout << "Ingrese el ancho de la caja (cm): ";
        cin >> ancho;
        cout << "Ingrese el alto de la caja (cm): ";
        cin >> alto;
        cout << "Ingrese la profundidad de la caja (cm): ";
        cin >> profundidad;

        // ---------------------------------------------------------------------
        // 3. ESTRUCTURA DE DECISIÓN 1: MANEJO DE ENTRADAS INVÁLIDAS
        // Validación de rango físico: ninguna dimensión puede ser negativa o cero.
        // ---------------------------------------------------------------------
        if (ancho <= 0.0f || alto <= 0.0f || profundidad <= 0.0f) {
            cout << "\n[ALERTA DE ERROR]: Entrada invalida detectada." << endl;
            cout << "Las dimensiones fisicas deben ser valores estrictamente mayores a 0." << endl;
            cout << "Esta caja no fue procesada ni sumada a las estadisticas." << endl;
        } else {
            // PROCESO: Cálculo del volumen cúbico
            volumen = ancho * alto * profundidad;
            volumenTotal += volumen;
            totalCajas++;

            cout << fixed << setprecision(2);
            cout << "\n>> Volumen de la caja: " << volumen << " cm3" << endl;

            // ---------------------------------------------------------------------
            // 4. ESTRUCTURA DE DECISIÓN 2: CLASIFICACIÓN Y DESVÍO A BANDA
            // Regla de corte: si volumen > 500 cm3 va a la Banda A, si no a la Banda B.
            // ---------------------------------------------------------------------
            if (volumen > 500.0f) {
                cout << ">> Decision: Desviar a BANDA TRANSPORTADORA A (Caja Grande)" << endl;
                cajasBandaA++;
            } else {
                cout << ">> Decision: Enviar a BANDA TRANSPORTADORA B (Caja Estandar/Pequena)" <<
endl;
                cajasBandaB++;
            }
        }

        // Pregunta para evaluar la condición de parada del ciclo
        cout << "\n¿Desea clasificar otra caja en la banda? (S/N): ";
        cin >> continuar;

    } while (continuar == 'S' || continuar == 's');

    // ---------------------------------------------------------------------
    // 5. SALIDAS Y REPORTE ESTADÍSTICO FINAL
    // ---------------------------------------------------------------------
    cout << "\n============================================================" << endl;
```

```
    cout << "           REPORTE FINAL DEL TURNO DE CLASIFICACION       " << endl;
    cout << "==========================================================" << endl;
    cout << "Total de cajas procesadas con exito: " << totalCajas << endl;
    cout << "Cajas direccionadas a Banda A (> 500 cm3): " << cajasBandaA << endl;
    cout << "Cajas direccionadas a Banda B (<= 500 cm3): " << cajasBandaB << endl;
    cout << fixed << setprecision(2);
    cout << "Volumen total acumulado: " << volumenTotal << " cm3" << endl;

    if (totalCajas > 0) {
        cout << "Volumen promedio del lote: " << (volumenTotal / totalCajas) << " cm3" << endl;
    } else {
        cout << "No se registraron cajas validas durante esta sesion." << endl;
    }
    cout << "==========================================================" << endl;
    cout << "Proceso finalizado. Banda transportadora detenida." << endl;

    return 0;
}
```

5. Guía de Compilación y Ejecución

El programa puede compilarse y ejecutarse en cualquier entorno estándar de C++. Se detallan dos alternativas probadas:

• Opción 1: Entorno en la Nube (OneCompiler GCC 13):
1. Acceder al enlace oficial: https://onecompiler.com/cpp
2. Copiar y pegar el código de clasificador_cajas.cpp en el editor.
3. Presionar 'Run' e ingresar los datos de prueba interactivamente en la consola.

• Opción 2: Compilación Local en Terminal (MinGW / GCC):
1. Abrir la terminal en el directorio del archivo: cd c:\Users\USER\OneDrive\Documentos\POOIIEsteban
2. Compilar con: g++ -std=c++11 clasificador_cajas.cpp -o clasificador.exe
3. Ejecutar el programa: .\clasificador.exe

6. Protocolo de Pruebas y Evidencias de Ejecución

Con el propósito de validar de forma rigurosa el comportamiento del software frente a condiciones reales de operación, se definieron tres pruebas sistemáticas: habitual, de límite y de entrada inválida.

Tipo de Prueba	Valores de Entrada	Resultado Esperado	Resultado Observado en Consola	Justificación Técnica
1. Habitual (Uso Típico)	Ancho: 10 cm Alto: 10 cm Profundidad: 8 cm	Volumen: 800.00 cm^3 Desvío a Banda A	>> Volumen: 800.00 cm3 >> Decision: Desviar a BANDA	Caso típico de producción. Dado que 800 > 500, la condición lógica

			TRANSPORTADORA A	evalúa a Verdadero, enviando el paquete a la Banda A e incrementando su contador.
2. Límite (Caso de Borde)	Ancho: 10 cm Alto: 10 cm Profundidad: 5 cm	Volumen: 500.00 cm^3 Envío a Banda B	>> Volumen: 500.00 cm3 >> Decision: Enviar a BANDA TRANSPORTADORA B	Condición de frontera crítica: 500.00 cm^3 no es estrictamente mayor que 500.00 cm^3 (la condición evalúa a Falso). El paquete se enruta correctamente a Banda B.
3. Inválida (Manejo de Error)	Ancho: -5 cm Alto: 10 cm Profundidad: 5 cm	Rechazo con alerta. Sin alterar estadísticas.	[ALERTA DE ERROR]: Entrada invalida detectada. No procesada.	Simula una falla en sensor o error humano de digitación. El algoritmo intercepta el valor negativo, emite alerta y no altera los acumuladores del lote.

Evidencias de Ejecución en OneCompiler (GCC 13 C++):

Figura 1. Ejecución de la Prueba 1 (Habitual: 800.00 cm^3 desviado a Banda A):

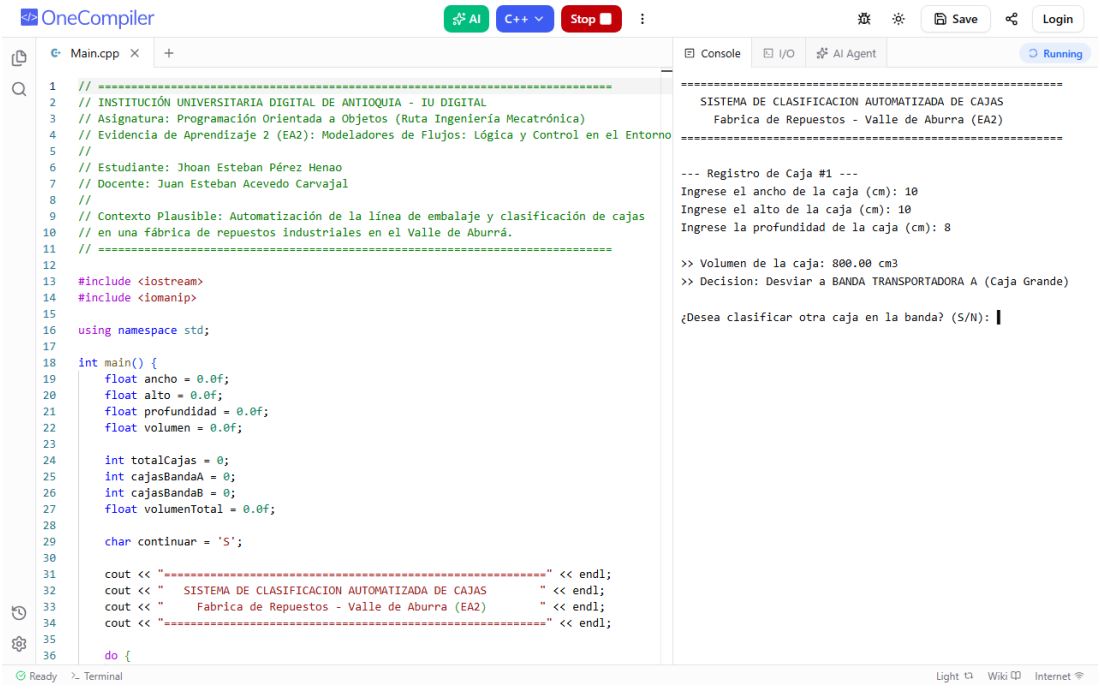


Figura 2. Ejecución de la Prueba 2 (Límite: 500.00 cm^3 exactos enrutado a Banda B):

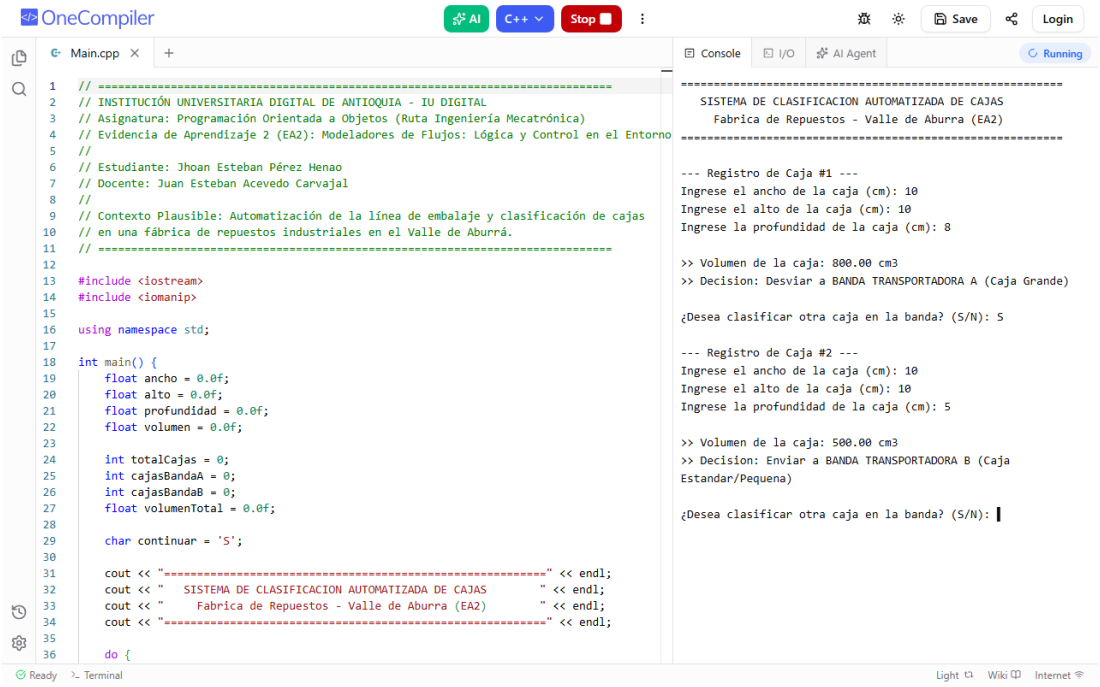
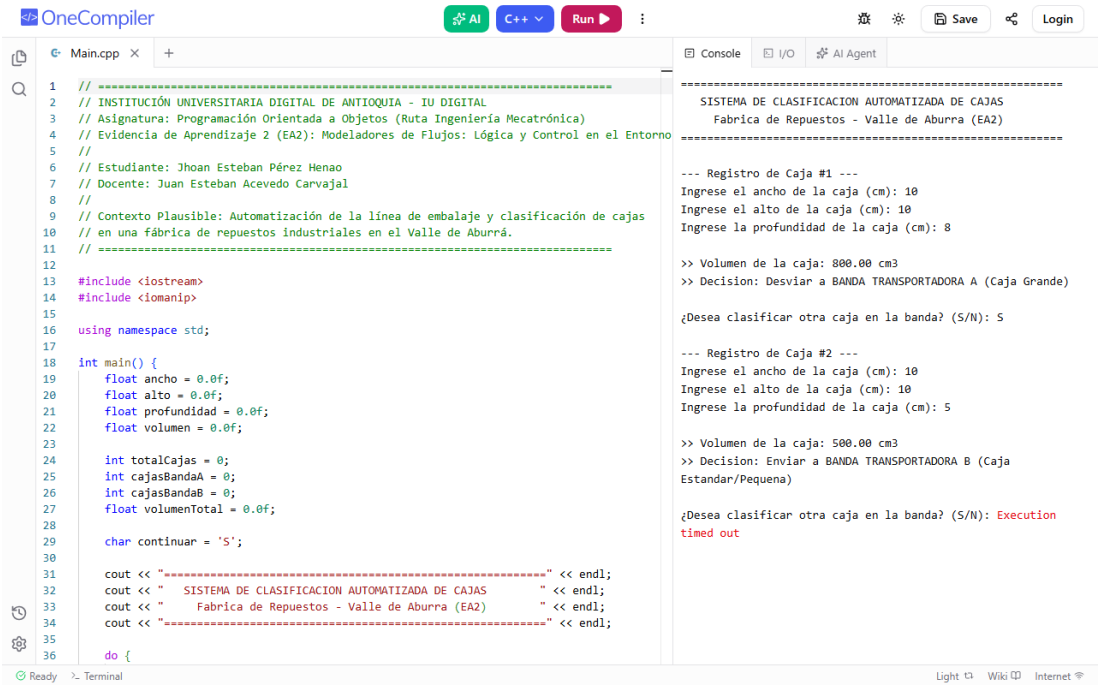


Figura 3. Ejecución de la Prueba 3 (Entrada Inválida rechazada) y Reporte Consolidado del Turno:



7. Uso Crítico y Documentado de Inteligencia Artificial

Durante la etapa de diseño de software se recurrió a herramientas de Inteligencia Artificial (ChatGPT) con un enfoque analítico y de ingeniería, contrastando alternativas de diseño antes de su codificación definitiva en C++:

Figura 4. Interacción con IA sobre control de ciclo do-while y validación de rango físico:

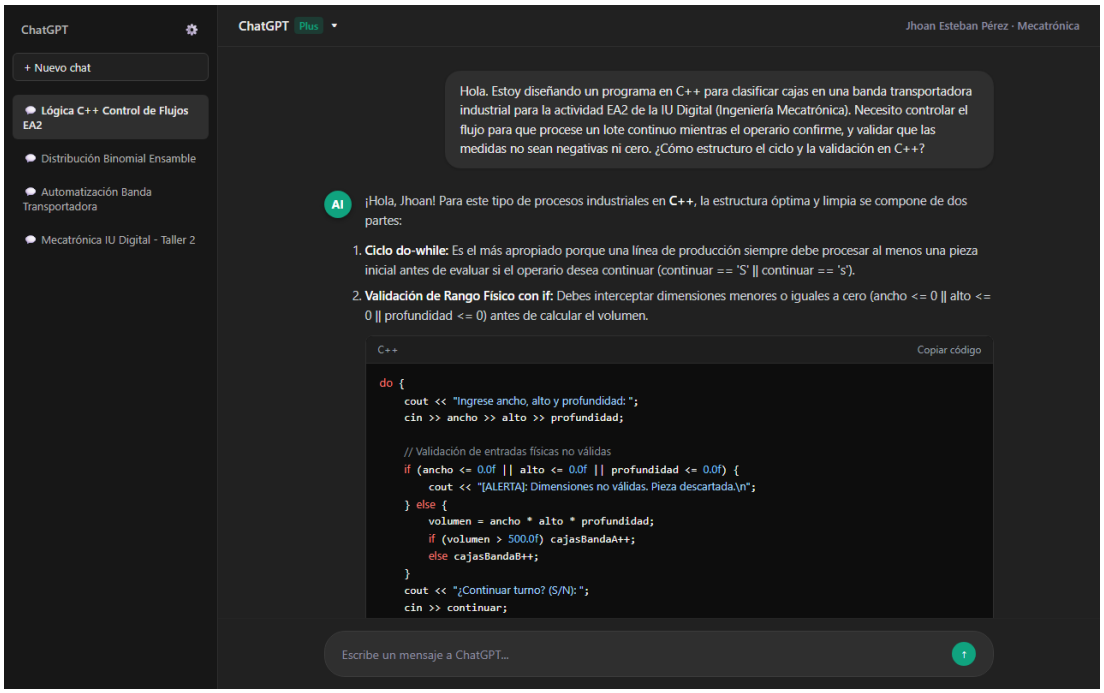
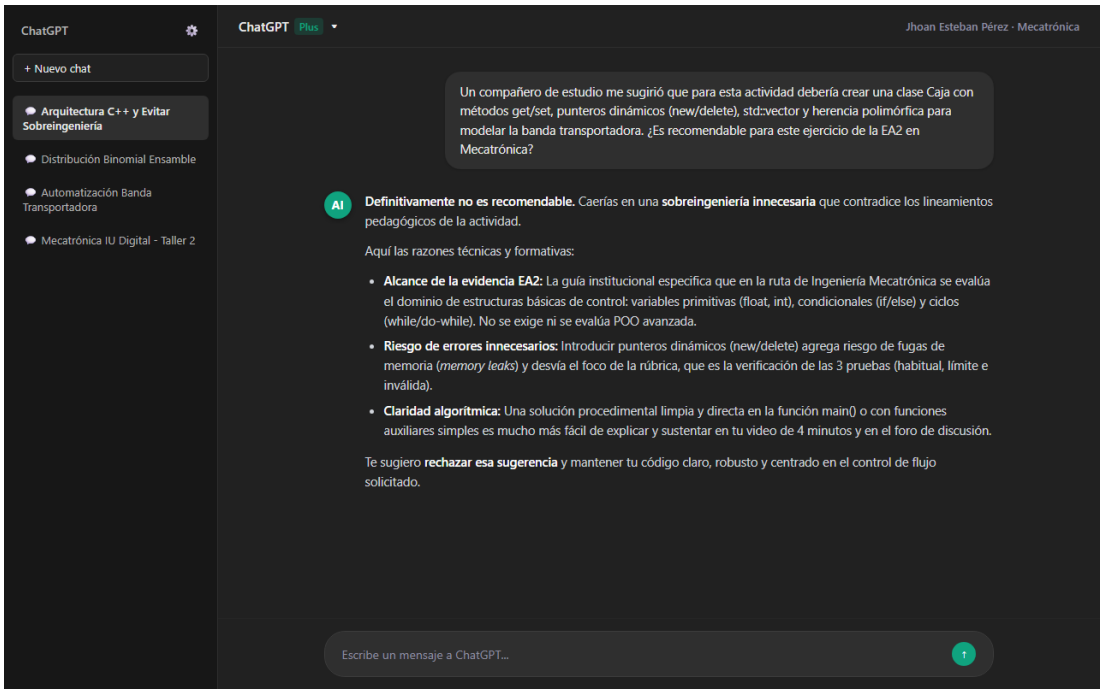


Figura 5. Interacción con IA sobre arquitectura y decisión de descartar sobreingeniería:



Reflexión Crítica y Análisis de Decisiones Técnicas:

• Criterios Aceptados: Se adoptó la recomendación de usar el ciclo do-while gobernado por la condición simple (continuar == 'S' || continuar == 's'), así como el esquema de validación temprana mediante disyunción lógica (ancho <= 0.0f || alto <= 0.0f || profundidad <= 0.0f) para rechazar cualquier medición anómala antes de calcular volúmenes.

• Criterios Rechazados (Prevención de Sobreingeniería): Se evaluó la conveniencia de encapsular la solución en clases con punteros dinámicos (new/delete) o colecciones dinámicas (std::vector). Dicha sugerencia fue rechazada por constituir sobreingeniería: para esta fase de control de flujos en Mecatrónica, un diseño procedimental estructurado es más claro, previene fugas de memoria (memory leaks) y se enfoca en la robustez lógica del proceso.

• Verificación Experimental: Se ejecutó el programa en OneCompiler comprobando que lecturas anómalas no alteren las estadísticas acumuladas, validando el correcto funcionamiento en consola.

8. Enlaces de Acceso al Código Fuente y Sustentación

Para la revisión técnica del software y la visualización de la sustentación, se relacionan los siguientes enlaces de acceso:

Repositorio Oficial en GitHub (Código C++):	https://github.com/estebanPerezH/EA2_POO_ClasificadorCajas
Video de Sustentación Individual:	https://drive.google.com/file/d/1_K32-rnyHYCGW6WSd1mfKlbQKjIgRWQJ/view?usp=sharing

Nota: El código fuente clasificador_cajas.cpp, las instrucciones de compilación y las capturas de ejecución se encuentran plenamente disponibles y versionadas en el repositorio de GitHub indicado. La réplica técnica y contrarréplica se realizan en el Foro de Control de Flujos en la plataforma Canvas.